Approximation Algorithm Notes

1 Definition

An **approximation algorithm** runs in polynomial time and finds solutions that are guaranteed to be close to optimal. We are interested in:

- Creating such algorithms using greedy algorithms or integer/linear programming
- Prove an approximation guarantee by comparing the approximated solution to the optimal solution

2 Greedy Algorithms

Coming up with a greedy approximtion algorithm is simple: first identify the objective (e.g. minimizing/maximizing a quantity), then design a simple local decision that aligns with this objective.

2.1 Proof

1. **Define the two solutions.** Define O, the optimal solution, and G, the solution obtained by the greedy algorithm.
2. **Describe the properties of O.** There should be some properties about O that you know without having to solve the problem. As two examples: if it is a partitioning problem, then O must be at least the average of the number of items you are partitioning over. If this is a matching problem, then O must be a valid matching, i.e. there is at most one instance of every candidate in O.
3. **Transform G.** For numerical problems, try to split up G in some way so that you can use the known properties of O to create an inequality that can be transformed to look like O. For other problems, try to consider what is different about G and O: why might O contain something that G missed? What does O have that G might not have?

2.2 Load Balancing Problem

Given a set of machines $M_1, \ldots, M_m$ and a set of n jobs with processing times $t_1, \ldots, t_n$, define:

- $A(i)$ is the set of jobs assigned to machine i
- The **load** on machine i is:
$$T_i = \sum_{j \in A(i)} t_j$$
- The **makespan** is the maximum load on any machine, i.e.
$$T = \max_i T_i$$

Our goal is to minimize the makespan, an NP-hard problem.

Algorithm. Initialize $T_i = 0$ for all i. Process each job one by one. For a job j, assign it to the machine that currently has the smallest load.

Proof (2-appox). Let O be the optimal makespan and G be the makespan found by the greedy algorithm. We hope to show that $2 \cdot O \geq G$. We know the following about O:

- It must be at least the average load per machine:
$$O \geq \frac{1}{m} \sum_j t_j$$

- It must be at least the maximum processing time, since we need some machine to process this job (so consequently, O is greater than all processing times):

$$O \geq \max_j t_j$$

Now, let M_i be the machine with the maximum load, so that $G = T_i$. Next, let j be the last job assigned to M_i. We can rewrite the total load on M_i to be:

$$G = T_i = (T_i - t_j) + t_j.$$

By the greedy algorithm, we know that before M_i processed job j, $(T_i - t_j)$ was the smallest load out of all the machines. That is, at the moment before j was assigned to M_i, $T_k \geq T_i - t_j$ for each machine k. Then, summing over all machines, we get that:

$$\sum_{k=1}^{m} T_k = \sum_{\ell=1}^{n} t_\ell \geq m \cdot (T_i - t_j).$$

Dividing both sides by m and adding t_j, we get:

$$t_j + \frac{1}{m} \sum_{\ell} t_\ell \geq T_i$$

Finally, using the two facts about O from above, we can see that O is greater than both terms on the LHS. Thus, it follows that $2 \cdot O \geq G$, as desired.

2.3 Set Cover Problem

Given a universe U, sets $S_1, \ldots, S_k$ with weights $w_1, \ldots, w_k$ such that

$$U = \bigcup_i S_i,$$

we wish to select some

$$I \subseteq \{1, \ldots, k\} \text{ such that } \bigcup_{i \in I} S_i = U \text{ and } \sum_{i \in I} w_i \text{ is minimized.}$$

We essentially have two objectives: (1) maximize the number of covered elements and (2) minimize the total weight. Our greedy heuristic will reflect this.

Algorithm. Let R be the set of uncovered elements and I be the indices of the selected sets. Initialize $R = U$ and $I = \emptyset$. While $R \neq \emptyset$, choose the set S_i with the lowest weight per covered element:

$$i = \arg\min_i \frac{w_i}{|S_i \cap R|}.$$

Proof. Let O be the optimal set cover and G be the set cover found by the greedy algorithm. First, we define the "cost" of an element when covered by a set S_i to be:

$$\text{cost}(u) = \frac{w_i}{|S_i \cap R|}.$$

Note that based on our greedy heuristic, the total cost of G is:

$$\sum_{i \in G} w_i = \sum_{u \in U} \text{cost}(u).$$

We now consider the cost for each set: the worst case occurs when each element in S_i is covered one by one, that is, the first element will have cost at most $\frac{w_i}{|S_i|}$, the second with cost at most $\frac{w_i}{|S_i|-1}$, and so forth, with the last element costing at most w_i. Using this, we can upper bound the total cost of S_i:

$$\sum_{u \in S_i} \text{cost}(u) \leq w_i(\frac{1}{|S_i|} + \frac{1}{|S_i| - 1} + ... + 1).$$

The summation on the RHS matches the harmonic series, which we can approximate with the natural log. Let $d = \max_i |S_i|$. Then:

$$H_d = 1 + \frac{1}{2} + \frac{1}{3} + ... + \frac{1}{d} \approx \ln d.$$

Finally, we can use H_d and the fact from above to bound our total cost of G:

$$\sum_{i \in G} w_i = \sum_{u \in S_i} \text{cost}(u) \leq w_i \cdot H_d$$

$$\sum_{i \in G} w_i \leq \sum_{i \in I} w_i \cdot H_d \approx \ln(\max_i |S_i|) \cdot \sum_{i \in I} w_i.$$

3 Linear Programming

A **linear program** (LP) consists of the variables $x_1, \ldots, x_n$, constraints $Ax \geq 0$ and $x \geq 0$, and the objective function

$$\min \sum_{i=1}^{n} c_i x_i,$$

where each x_i contributes c_i to the cost (i.e. we wish to minimize the total cost).

An **integer program** (IP) is the same as an LP, except we require that each $x_i \in \{0, 1\}$. IP is NP-hard.

Given some hard problem, we first write it as an IP. To approximate the solution, we solve it as an LP (i.e. allowing fractional values). The solution found by LP is a lower bound for the optimal solution.

3.1 Weighted Vertex Cover Problem

Recall: A **vertex cover** of a graph $G = (V, E)$ is a set $S \subseteq V$ such that each edge has at least one end in S.

In the **weighted vertex cover** problem, given a graph $G = (V, E)$, with each vertex v_i associated with weight $w_i \geq 0$, our goal is to find a cover S that minimizes the total weight, i.e.

$$\min \sum_{i \in S} w_i.$$

Integer Program. Define a variable x_i for each vertex $v_i \in V$ with $x_i = 1$ if $v_i \in S$ and $x_i = 0$ otherwise. Following the constraints for the original problem, the IP is:

$$\begin{aligned}
&\text{minimize } \sum_{i=1}^{n} x_i w_i \\
&\text{subject to} \\
&\qquad x_i + x_j \geq 1 \qquad \text{for all } (v_i, v_j) \in E \\
&\qquad x_i \in \{0, 1\} \quad \text{for all } i = 1, ..., |V|
\end{aligned}$$

To solve this as an LP, simply relax constraint (2) to $0 \leq x_i \leq 1$.